

UNIVERSIDADE ESTADUAL DO PIAUÍ — UESPI

Centro de Tecnologia e Urbanismo — Bacharelado em Ciência da Computação

Disciplina: Sistemas Distribuídos — 3ª Avaliação

RELATÓRIO TÉCNICO — SISTEMA DISTRIBUÍDO DE PEDIDOS

Alunos: Filipe Lopes

Professor: Bringel Filho

Data: julho de 2026

1. Introdução

Este trabalho apresenta o projeto e a implementação de uma plataforma distribuída de pedidos on-line, inspirada em sistemas de comércio eletrônico. O objetivo é integrar os conceitos estudados na disciplina, com ênfase em arquitetura de microsserviços, comunicação remota e indireta, concorrência, sincronização e tolerância a falhas.

A solução foi desenvolvida em Node.js e executada em contêineres Docker. Ela é formada por quatro serviços independentes: Gateway, Estoque, Pedidos e Notificações. Além de um worker de publicação de eventos, dois bancos PostgreSQL isolados e um broker RabbitMQ. O cliente utiliza apenas a API pública do Gateway; as demais interações acontecem pela rede interna criada pelo Docker Compose.

O caso principal consiste em receber um pedido, reservar o produto sem permitir estoque negativo, persistir a compra, publicar o evento ``pedido_criado`` e processar uma

notificação. Se a gravação do pedido falhar depois da reserva, o sistema tenta compensar a operação, devolvendo a quantidade ao estoque.

2. Arquitetura distribuída

A aplicação adota arquitetura cliente-servidor em múltiplas camadas, organizada segundo o estilo de microsserviços. Cada componente possui processo, responsabilidade, dependências e porta próprios. O isolamento reduz o acoplamento e permite que serviços e infraestrutura sejam substituídos ou escalados separadamente.

Cliente

|

| HTTP POST /pedido

v

Gateway :3000

|----- HTTP POST /reservar -----> Estoque :3001

|

|

|

v

|

PostgreSQL Estoque

|

|----- HTTP POST /criar -----> Pedidos :3002

|

v

PostgreSQL Pedidos

(compras + outbox)

|

v

Worker Outbox

|

| AMQP

v

RabbitMQ

|

v

Notificações

Os contêineres compartilham a rede bridge `uespi_network`. A descoberta de serviços é realizada pelo DNS interno do Docker, utilizando nomes como ``estoque``, ``pedidos``, ``estoque_db``, ``pedidos_db`` e ``rabbitmq``. As URLs são fornecidas por variáveis de ambiente, evitando endereços fixos no código para a execução containerizada.

Os dados seguem o princípio de banco por serviço. O Estoque é proprietário das tabelas ``produtos`` e ``reservas``, enquanto Pedidos mantém ``compras`` e ``outbox_events`` em outro PostgreSQL. Nenhum serviço consulta diretamente o banco do outro. Essa separação favorece autonomia, embora impeça uma transação ACID única entre todo o fluxo, razão pela qual são empregadas compensação e mensageria confiável.

3. Serviços e responsabilidades

3.1 API Gateway

O Gateway, exposto na porta 3000, representa o único ponto de entrada público. A rota ``POST /pedido`` valida se ``produto`` é uma string não vazia e se ``quantidade`` é um inteiro positivo. Em seguida, coordena duas invocações REST: primeiro reserva o item no Estoque e depois solicita a criação da compra ao serviço de Pedidos.

As chamadas internas possuem timeout de três segundos. O Gateway também valida o contrato das respostas e converte erros internos em códigos HTTP adequados: 400 para entrada inválida, 404 para produto inexistente, 409 para conflito ou estoque insuficiente, 502 para resposta inválida e 503 para timeout ou serviço indisponível.

Se o Estoque já tiver confirmado a reserva e a criação do pedido falhar, o Gateway executa a ação compensatória `POST /cancelar-reserva`. Essa coordenação corresponde a uma Saga simples, orquestrada pelo Gateway.

3.2 Serviço de Estoque

O Estoque, na porta 3001, mantém a quantidade disponível dos produtos e o histórico das reservas. A rota `POST /reservar` inicia uma transação PostgreSQL, seleciona o produto com `SELECT ... FOR UPDATE`, verifica a disponibilidade, reduz o saldo e insere uma reserva ativa. A redução e o registro da reserva são confirmados no mesmo `COMMIT`; qualquer erro provoca `ROLLBACK`.

A rota `POST /cancelar-reserva` também usa transação e bloqueio pessimista sobre a reserva. Se ela estiver ativa, a quantidade é devolvida ao produto e o estado muda para `CANCELADA`. Se já estiver cancelada, a chamada retorna sucesso sem somar novamente a quantidade. Portanto, a compensação é idempotente e segura contra cancelamentos concorrentes.

3.3 Serviço de Pedidos

O serviço de Pedidos, na porta 3002, valida os dados recebidos e grava a compra. Na mesma transação local, insere também uma representação do evento `pedido_criado` na tabela `outbox_events`. O uso do padrão Transactional Outbox elimina a janela em que uma compra poderia ser confirmada no banco sem existir um evento persistido correspondente.

3.4 Worker Outbox

O worker é um processo independente que consulta eventos cujo campo ``publicado_em`` ainda está nulo. A consulta utiliza ``FOR UPDATE SKIP LOCKED``, permitindo a existência de mais de um worker sem que dois deles selecionem simultaneamente a mesma linha.

O evento é enviado para a fila durável ``fila_notificacoes`` como mensagem persistente. O canal de confirmação do RabbitMQ e ``waitForConfirms()`` são utilizados antes de marcar o evento como publicado. Se o broker estiver indisponível ou não confirmar a mensagem, ocorre rollback no banco, o evento continua pendente e será tentado novamente.

3.5 Serviço de Notificações

O serviço de Notificações não expõe API HTTP. Ele consome mensagens do RabbitMQ e simula o envio de e-mail por meio do log. Depois do processamento, envia ACK manual, permitindo que o broker remova a mensagem da fila.

Um conjunto em memória armazena os identificadores já processados para evitar notificações repetidas durante a mesma execução. Essa solução demonstra idempotência, mas não é permanente: após reiniciar o consumidor, os identificadores são perdidos. Em produção, seria recomendável registrar o ``messageId`` em banco com restrição de unicidade.

4. Comunicação entre componentes

A comunicação síncrona usa REST sobre HTTP/TCP, com corpos JSON. Ela ocorre entre cliente e Gateway e entre Gateway, Estoque e Pedidos. Esse modelo oferece contratos simples e respostas imediatas para as etapas necessárias à confirmação do pedido.

A comunicação assíncrona usa AMQP e RabbitMQ. Pedidos não chama Notificações diretamente: publica um evento por meio do worker Outbox, e o consumidor o processa de forma desacoplada. Assim, o pedido pode ser confirmado mesmo se Notificações estiver temporariamente indisponível; a mensagem permanece na fila durável até que um consumidor esteja disponível.

O sistema adota serialização JSON nos dois modelos. A separação entre comunicação síncrona e indireta é intencional: reserva e persistência precisam compor a resposta ao cliente, enquanto a notificação pode ser processada posteriormente.

5. Sincronização e controle de concorrência

O problema de concorrência avaliado é a compra simultânea da última unidade. Sem sincronização, duas requisições poderiam ler saldo 1, aprovar as duas compras e produzir saldo negativo. A solução utiliza exclusão mútua no nível da linha do PostgreSQL.

Quando uma transação executa ``SELECT ... FOR UPDATE`` sobre um produto, outra transação que tente reservar o mesmo produto aguarda a liberação do bloqueio. A primeira reduz o saldo e confirma. Em seguida, a segunda lê o valor atualizado e recebe HTTP 409 se a quantidade for insuficiente. Produtos diferentes podem ser processados paralelamente, pois o bloqueio não abrange toda a tabela.

Há sincronização semelhante no cancelamento de reservas. Além disso, ``FOR UPDATE SKIP LOCKED`` ordenado pelo identificador da Outbox coordena workers concorrentes. O identificador sequencial dos eventos oferece uma ordem local de publicação no banco de Pedidos; entretanto, o sistema não implementa relógio lógico global, nem promete ordenação total entre múltiplos produtores.

6. Tolerância a falhas e consistência

As principais estratégias implementadas são:

- timeout de três segundos nas invocações feitas pelo Gateway;
- tradução de indisponibilidade e timeout para HTTP 503;
- rollback de transações locais em caso de erro;
- compensação idempotente da reserva quando Pedidos falha;
- persistência atômica da compra e do evento com Transactional Outbox;

- repetição posterior da publicação quando RabbitMQ está indisponível;
- fila durável, mensagens persistentes, publisher confirms e ACK do consumidor;
- proteção simples contra notificação duplicada durante a vida do processo.

O modelo de entrega é, na prática, pelo menos uma vez. Existe uma pequena janela entre o RabbitMQ confirmar a publicação e o worker gravar `publicado\_em`: se o worker falhar nesse intervalo, o mesmo evento pode ser republicado. Por isso, o consumidor deve permanecer idempotente.

A Saga também possui uma limitação: se Pedidos falhar e, ao mesmo tempo, o Estoque estiver indisponível durante a compensação, o Gateway apenas registra um alerta crítico. Não há fila persistente para repetir a compensação. Uma evolução seria criar uma outbox de comandos compensatórios ou um coordenador de Saga persistente.

7. Testes e demonstração

Os seguintes cenários atendem à demonstração solicitada no enunciado. Antes dos testes, o ambiente deve ser iniciado com `docker compose up --build`.

7.1 Fluxo normal

Enviar `POST /pedido` com `{ "produto": "Notebook", "quantidade": 1 }`. O resultado esperado é HTTP 201, com `pedidoId` e estado `CONCLUIDO`. Os logs devem mostrar a reserva, a gravação da compra, a publicação do evento e a simulação do e-mail. Também devem existir uma compra, uma reserva ativa e um evento Outbox marcado como publicado.

7.2 Concorrência sobre a última unidade

Definir o saldo de Notebook como 1 e enviar duas requisições simultâneas. O resultado esperado é exatamente uma resposta 201 e uma resposta 409. A consulta posterior ao banco deve mostrar saldo 0, nunca negativo, comprovando o efeito de `FOR UPDATE`.

7.3 Serviço de Pedidos fora do ar

Interromper o contêiner `pedidos` e enviar uma compra com estoque disponível. O Gateway deve responder 503 e chamar o cancelamento. Ao consultar o banco de Estoque, a reserva deve estar `CANCELADA` e a quantidade deve ter sido devolvida.

7.4 Atraso de rede ou timeout

Introduzir atraso superior a três segundos em uma resposta interna, ou pausar temporariamente o serviço chamado. O Gateway deve encerrar a espera e responder 503. Caso a reserva já tenha sido confirmada antes do atraso em Pedidos, deve ser iniciada a compensação.

Esse cenário evidencia uma ambiguidade inerente a timeouts: Pedidos pode concluir após o Gateway desistir, enquanto o Gateway cancela a reserva. Para maior robustez, a criação de pedidos deveria aceitar uma chave de idempotência e possuir consulta de estado antes da compensação.

7.5 RabbitMQ indisponível

Interromper o RabbitMQ antes de criar um pedido. A compra e o evento devem permanecer gravados, com `publicado\_em` nulo. Após reiniciar o broker, o worker deve reconectar, publicar o evento e preencher o instante de publicação, sem perder a notificação.

7.6 Mensagem duplicada

Publicar duas vezes o mesmo evento ou provocar uma republicação antes da atualização da Outbox. Durante a mesma execução do consumidor, o primeiro evento simula o e-mail e o segundo é reconhecido como já processado. Após reiniciar Notificações, essa garantia

deixa de existir devido ao armazenamento em memória, limitação que deve ser mencionada durante a apresentação.

8. Atendimento aos requisitos e conclusão

A aplicação satisfaz os cinco requisitos obrigatórios do trabalho e demonstra conceitos fundamentais de sistemas distribuídos: separação de responsabilidades, comunicação em rede, bancos isolados, concorrência controlada e tratamento de falhas parciais. A combinação de transações locais, Saga compensatória e Transactional Outbox oferece consistência adequada para uma solução acadêmica sem recorrer a transações distribuídas globais.

Como melhorias futuras, destacam-se a persistência da idempotência do consumidor, a repetição durável de compensações, chaves idempotentes na criação de pedidos, health checks no Docker Compose, autenticação, observabilidade distribuída e testes automatizados de integração e falhas. Essas evoluções aproximariam o protótipo das exigências de confiabilidade de uma plataforma em produção.

Referências

- Enunciado “3ª avaliação”, disciplina Sistemas Distribuídos, UESPI.
- Documentação do projeto e código-fonte da aplicação.
- Documentação oficial do PostgreSQL: transações e bloqueios de linha.
- Documentação oficial do RabbitMQ: filas duráveis, acknowledgements e publisher confirms.